

Модуль 3. “Регулярні мови та вирази”

дисципліни

“Розробка компіляторів для предметно-орієнтованих мов”

Лектор: *проф. Григорій ЖОЛТКЕВИЧ, д.т.н.*

Зміст

1	Що таке формальна мова?	1
2	Синтаксис та семантика мови	1
2.1	Синтаксис: Форма та Структура	1
2.2	Семантика: Зміст та Значення	2
2.3	Зв'язок між синтаксисом і семантикою	2
3	Як визначити синтаксис мови?	3
3.1	Ланцюжки символів	3
3.2	Мови та операції над ними	5
4	Похідна Бжозовськи мови. Критерій належності ланцюжка мові	5
4.1	Визначення похідної Бжозовськи. Правила диференціювання	5
4.2	Похідні вищих порядків. Ланцюгове правило	6
4.3	Критерій належності ланцюжка мові	6
5	Регулярні вирази і регулярні мови	7
5.1	Основні поняття	7
5.2	Спрощення регулярних виразів	8
5.3	Диференціювання регулярних виразів за Бжозовськи	8

Ця тема є теоретичною основою синтаксичного аналізу – тут вводяться необхідні визначення та позначення, а також подається огляд теорії регулярних мов.

1 Що таке формальна мова?

Почнемо з уточнення поняття формальної мови.

Формальна мова — це множина *скінченних ланцюжків символів*, побудованих з елементів певного скінченного алфавіту.

У математиці, логіці та інформатиці *формальна мова* не є “природною” мовою, як українська чи англійська, а є точно визначеною системою. Вона складається з двох ключових елементів:

1. **алфавіту** (*alphabet*) – скінченної, непорожньої множини символів. Наприклад, алфавітом можуть бути $\{0, 1\}$, $\{a, b, c\}$, або $\{+, *, (,)\}$;
2. **множини ланцюжків** (*set of chains*) – набору усіх *допустимих* (*well-formed*) ланцюжків (послідовностей скінченної довжини) сформованих з символів цього алфавіту.

Приклад 1. Наведемо простий приклад формальної мови. Вона складається з

- **алфавіту** $\{0, 1\}$ та
- **множини ланцюжків**, що включає усі ланцюжки, які починаються з 1 і мають парну кількість 0.

Ця мова **включає** такі ланцюжки, як 1, 100, 1100, 1010 тощо. А от ланцюжки 01, 10, 111 **не належать** до цієї мови.

Концепція формальної мови є основою для визначення синтаксису мов програмування, протоколів зв'язку та багатьох інших систем. Точність цієї концепції дозволяє комп'ютерним системам однозначно інтерпретувати інструкції, уникаючи двозначності, притаманної природним мовам.

2 Синтаксис та семантика мови

Будь-яка мова, чи то природна, як-от українська, чи штучна, як мова програмування Python, має два фундаментальні аспекти: **синтаксис** і **семантику**. Розуміння цих концепцій є ключовим для правильного використання мови та її інтерпретації.

2.1 Синтаксис: Форма та Структура

Синтаксис мови визначається її **граматичними правилами** – скінченим набором інструкцій, які вказують, як правильно поєднувати символи, слова та інші елементи, щоб утворити коректні мовні конструкції. Іншими словами, синтаксис – це про **форму**, а не про зміст.

Для людської мови синтаксичні правила регулюють порядок слів у реченні, узгодженість відмінків, родів, часів тощо. Наприклад, в українській мові речення “*Котик ловить мишку*” є синтаксично правильним, тоді як “*Котик ловить мишка*” – синтаксично некоректне.

У мовах програмування синтаксис набагато суворіший. Компілятор або інтерпретатор не може здогадатися, що ви мали на увазі. Наприклад, у мові C++ синтаксис вимагає ставити крапку з комою ‘;’ після кожної інструкції:

```
int x = 5;
int y = 10;
```

Лістинг 1: Приклад мовою C++

Якщо ви забудете поставити ‘;’ після першого рядка, компілятор видасть синтаксичну помилку, оскільки порушено синтаксичне правило мови.

2.2 Семантика: Зміст та Значення

На відміну від синтаксису, **семантика** визначає **значення** або **зміст** конструкцій, які є синтаксично правильними. Це про те, що саме робить програма, коли вона виконується.

Повернімося до прикладу з людською мовою. Речення “Безколісний автомобіль швидко доїхав до місяця” є синтаксично правильним (підмет, присудок, додатки стоять на своїх місцях), але семантично безглуздим, оскільки його зміст не відповідає реальності.

У мовах програмування семантика пов’язана з тим, як виконується код. Синтаксично правильний вираз може мати різну семантику. Наприклад, залежно від типів змінних, оператор ‘+’ може означати як арифметичне додавання, так і конкатенацію рядків.

```
x = 5
y = 10
result_int = x + y
# Семантика: додавання чисел, результат 15

a = "Hello"
b = " World"
result_str = a + b
# Семантика: конкатенація рядків, результат "Hello World"
```

Лістинг 2: Приклад мовою Python

Навіть якщо програма є синтаксично правильною, вона може містити **семантичні помилки**.

Приклад 2 (ділення на нуль). Це не синтаксична, а семантична помилка. Код `x = 5 / 0` є синтаксично коректним, але його виконання призведе до помилки, оскільки операція не має визначеного значення.

Приклад 3 (нескінченний цикл). Конструкція є синтаксично правильною, але з точки зору семантики може бути небажаною, оскільки програма ніколи не завершить свою роботу.

У комп’ютерних науках існують різні підходи до визначення семантики мов.

Найбільш інтуїтивно зрозумілим є **операційний підхід**. Операційна семантика описує, як виконання програми змінює стан обчислювальної системи, крок за кроком. Тому цю семантику називають ще “**семантикою малих кроків**”.

На відміну від операційної семантики **денотаційна семантика** співставляє певне математичне значення кожній мовній конструкції.

Аксіоматична семантика використовує логічні твердження для опису властивостей програми і, в певному розумінні, є граничною формалізацією денотаційної семантики.

2.3 Зв’язок між синтаксисом і семантикою

Синтаксис і семантика нерозривно пов’язані, як форма і зміст. Синтаксис є фундаментом, на якому будується семантика.

Синтаксично некоректна конструкція не має семантики. Наприклад, $5 + *$ не є виразом, тому ми не можемо говорити про його значення. Спочатку програма має бути синтаксично правильною, і лише після цього можна визначити її семантику.

На етапі компіляції чи інтерпретації, першим кроком завжди є *синтаксичний аналіз*. Парсер перетворює послідовність символів у структуроване представлення – *абстрактне синтаксичне дерево (AST)*. Це дерево є тим “містком”, що з’єднує синтаксичну структуру з семантичним змістом. Наступний етап – *семантичний аналіз* – перевіряє логічну коректність програми та визначає, як її виконати.

Наприклад, якщо ми маємо AST для виразу $(+ (* 3 10) 5)$, то

- о синтаксис забезпечує, що дужки та операції розставлені правильно, а
- о семантика визначає, що $*$ означає множення, а $+$ – додавання.

Отже, синтаксис – це правила побудови, а семантика – це правила означування. Без синтаксису не існує виразу, а без семантики – його значення.

Усі мови, від людських до комп’ютерних, існують і функціонують завдяки єдності цих двох мовних аспектів.

3 Як визначити синтаксис мови?

Як вже було сказано, формальна мова є підмножиною ланцюжків символів певного алфавіту, тож синтаксичні правила мови забезпечують специфікацію цієї множини. Отже, синтаксичні правила є засобом розрізнення допустимих та недопустимих ланцюжків. Давайте формалізуємо необхідні для подальшого поняття.

3.1 Ланцюжки символів

Нагадаємо, що визначення мови починається з визначення її алфавіту. Тому будемо вважати, що такий алфавіт вже визначений, позначивши його через Σ . Зазвичай елементи такого алфавіту називають *термінальними символами* або коротко – *терміналами*.

Що ж ми розуміємо під терміном “ланцюжок”?

Визначення 1 (ланцюжка рекурсивно). Ланцюжком є

- 1) *порожній ланцюжок* (ланцюжок без символів), який позначається $\langle \rangle$;
- 2) ланцюжок $a \cdot u$, побудований шляхом *вставлення символу* $a \in \Sigma$ в *голову вже побудованого ланцюжка* u ;
- 3) ніяких інших ланцюжків, окрім побудованих за правилами 1 та 2, немає.

Це визначення відповідає на питання, як об’єкт утворюється шляхом покрокового застосування правил. Воно дозволяє побудувати всі можливі ланцюжки з термінальних символів.

Виходячи з цього визначення, кожний ланцюжок має вигляд

- або $\langle \rangle$,
- або $a_1 \cdot (\dots (a_n \cdot \langle \rangle) \dots)$ для деякого $n > 0$ та $a_1, \dots, a_n \in \Sigma$, що коротко записується у формі $\langle a_1, a_2, \dots, a_n \rangle$.

Зауважимо, що множину ланцюжків, елементи яких належать алфавіту Σ , зазвичай позначають через Σ^* .

Довжина ланцюжка є функцією $u \in \Sigma^* \mapsto \text{length}(u) \in \mathbb{N}$, яка співставляє ланцюжку u його довжину; далі позначається через $|u|$:

```
def length( $u: \Sigma^*$ )  $\rightarrow \mathbb{N}$ :
```

```
    if  $u == \langle \rangle$ :
        return 0
    #  $u == a \cdot u'$ 
    return 1 + length( $u'$ )
```

Функція доступу до k -ого елементу ланцюжка є (частковою) функцією, що визначається рекурсивно:

```
def take( $u: \Sigma^*, k: \mathbb{N}$ )  $\rightarrow \Sigma \mid \text{None}$ :
```

```
    if  $|u| < k$ :
        return None
    #  $u == a \cdot u'$ 
    if  $k == 0$ :
        return  $a$ 
    return take( $u', k - 1$ )
```

Операція конкатенації ланцюжків визначається рекурсивно:

```
def concat( $u: \Sigma^*, v: \Sigma^*$ ):
```

```
    if  $u == \langle \rangle$ :
        return  $v$ 
    #  $u == a \cdot u'$ 
    return  $a \cdot \text{concat}(u', v)$ 
```

Вправа 1. Доведіть, що множина Σ^* з операцією конкатенації та ланцюжком $\langle \rangle$ як одиниця є моноїдом, тобто

1. для будь-яких $u, v, w \in \Sigma^*$ вірною є рівність $u \cdot (v \cdot w) = (u \cdot v) \cdot w$;
2. для будь-якого u вірною є рівність $u \cdot \langle \rangle = \langle \rangle \cdot u = u$.

Вправа 2. Доведіть, що функція $|\cdot|: \Sigma^* \rightarrow \mathbb{N}$ є гомоморфізмом моноїда $(\Sigma^*, \cdot, \langle \rangle)$ у моноїд $(\mathbb{N}, +, 0)$, тобто

1. $|\langle \rangle| = 0$;
2. для будь-яких $u, v \in \Sigma^*$ вірною є рівність $|u \cdot v| = |u| + |v|$.

3.2 Мови та операції над ними

Як вже було сказано, формальною мовою з алфавітом Σ є будь-яка підмножина $L \subset \Sigma^*$.

Таким чином, ми можемо говорити про

- порожню мову $\emptyset$;
- тривіальну мову $E = \{\langle \rangle\}$;
- однолітерні мови $L_a = \{\langle a \rangle\}$, де $a \in \Sigma$.

Оскільки мови є множинами, то до них можна застосовувати відповідні операції:

- $L_1 \cup L_2$, $L_1 \cap L_2$, де $L_1, L_2 \subset \Sigma^*$;
- $\bar{L} = \Sigma^* \setminus L$, де $L \subset \Sigma^*$.

Крім цих операцій, є специфічні для мов операції:

- операція **конкатенації** мови L_1 з мовою L_2 :

$$L_1 \cdot L_2 = \{u \in \Sigma^* \mid (\exists u' \in L_1, u'' \in L_2) u = u' \cdot u''\}.$$

Неформально кажучи, $u \in L_1 \cdot L_2$, якщо u можна представити у вигляді конкатенації його префікса і суфікса так, що перший з них належить L_1 , а другий – L_2 ;

- операція **замкнення Кліні** мови L :

$$L^* = \bigcup_{n \geq 0} L^n, \quad \text{де } L^0 = \{\langle \rangle\}, \quad L^{n+1} = L \cdot L^n.$$

Неформально кажучи, $u \in L^*$, якщо або $u = \langle \rangle$, або $u \in L$, або $u = u_1 \cdot \dots \cdot u_k$ з $k \geq 2$ так, що кожний $u_i \in L$.

4 Похідна Бжозовськи мови. Критерій належності ланцюжка мові

Сенс введення похідних Бжозовськи полягає у тому, як ми побачимо далі, що техніка, яка базується на похідних, спрощує вирішення задачі про розпізнавання належності ланцюжка $u \in \Sigma^*$ мові $L \subset \Sigma^*$.

4.1 Визначення похідної Бжозовськи. Правила диференціювання

Похідна Бжозовськи $a^{-1} \cdot L$ мови $L \subset \Sigma^*$ відносно символу $a \in \Sigma$ є мовою, що визначається формулою

$$a^{-1} \cdot L = \{u \in \Sigma^* \mid a \cdot u \in L\}.$$

Твердження 1 (правила диференціювання). Нехай $L, M \subset \Sigma^*$ та $a, b \in \Sigma$, тоді

1. $a^{-1} \cdot \emptyset = a^{-1} \cdot E = \emptyset$;
2. $a^{-1} \cdot L_b = \begin{cases} E & \text{якщо } a = b \\ \emptyset & \text{якщо } a \neq b \end{cases}$;
3. $a^{-1} \cdot (L^*) = (a^{-1} \cdot L) \cdot L^*$;

$$4. a^{-1} \cdot (L \cdot M) = \begin{cases} (a^{-1} \cdot L) \cdot M & \text{якщо } \langle \rangle \notin L \\ (a^{-1} \cdot L) \cdot M \cup a^{-1} \cdot M & \text{якщо } \langle \rangle \in L \end{cases}$$

$$5. a^{-1} \cdot (L \cup M) = a^{-1} \cdot L \cup a^{-1} \cdot M.$$

Доведення п. 4. Припустимо, що $\mathbf{u} \in (a^{-1} \cdot L) \cdot M$, що еквівалентно можливості декомпонувати $\mathbf{u} = \mathbf{v} \cdot \mathbf{w}$ так, щоб $\mathbf{v} \in a^{-1} \cdot L$, $\mathbf{w} \in M$. Інакше кажучи, маємо $a \cdot \mathbf{u} = a \cdot (\mathbf{v} \cdot \mathbf{w}) = (a \cdot \mathbf{v}) \cdot \mathbf{w}$. Враховуючи $\mathbf{v} \in a^{-1} \cdot L$, маємо $a \cdot \mathbf{v} \in L$, тобто $a \cdot (\mathbf{v} \cdot \mathbf{w}) \in L \cdot M$. Останнє гарантує $(a^{-1} \cdot L) \cdot M \subset a^{-1} \cdot (L \cdot M)$. $\square$

Вправа 3. Доведіть пп. 1–3, 5 Твердження 1.

4.2 Похідні вищих порядків. Ланцюгове правило

Щоб продемонструвати як це забезпечується, введемо похідні Бжозовські вищих порядків:

1. погодимося, що $\langle \rangle^{-1} \cdot L = L$ для будь-якої мови $L \subset \Sigma^*$;
2. визначимо $\langle a \rangle^{-1} \cdot L = a^{-1} \cdot L$ для будь-якої мови $L \subset \Sigma^*$ та символу $a \in \Sigma$;
3. визначимо також $(a \cdot \mathbf{u})^{-1} \cdot L = \mathbf{u}^{-1} \cdot (a^{-1} \cdot L)$ для будь-якої мови $L \subset \Sigma^*$, символу $a \in \Sigma$ та ланцюжка $\mathbf{u} \in \Sigma^*$.

Твердження 2. Для будь-якої мови $L \subset \Sigma^*$ та ланцюжка $\mathbf{u} \in \Sigma^*$

$$\mathbf{u}^{-1} \cdot L = \{\mathbf{w} \in \Sigma^* \mid \mathbf{u} \cdot \mathbf{w} \in L\}.$$

Доведення. Скористаємося принципом індукції, для чого зауважимо

1. у випадку $\mathbf{u} = \langle \rangle$ рівність є, очевидно, вірною;
2. якщо рівність є вірною для деякого $\mathbf{u} \in \Sigma^*$, тоді

$$\mathbf{w} \in (a \cdot \mathbf{u})^{-1} \cdot L \iff \mathbf{w} \in \mathbf{u}^{-1} \cdot (a^{-1} \cdot L),$$

тобто за припущенням маємо

$$\mathbf{w} \in (a \cdot \mathbf{u})^{-1} \cdot L \iff \mathbf{u} \cdot \mathbf{w} \in a^{-1} \cdot L \iff a \cdot (\mathbf{u} \cdot \mathbf{w}) \in L \iff (a \cdot \mathbf{u}) \cdot \mathbf{w} \in L,$$

що означає

$$(a \cdot \mathbf{u})^{-1} \cdot L = \{\mathbf{w} \in \Sigma^* \mid (a \cdot \mathbf{u}) \cdot \mathbf{w} \in L\}.$$

Скориставшись принципом індукції отримаємо необхідний висновок. $\square$

4.3 Критерій належності ланцюжка мові

Твердження 3. Для будь-якої мови $L \subset \Sigma^*$ та ланцюжка $\mathbf{u} \in \Sigma^*$

$$\mathbf{u} \in L \quad \text{тоді і тільки тоді, коли} \quad \langle \rangle \in \mathbf{u}^{-1} \cdot L.$$

Доведення. Дійсно, у випадку $\mathbf{u} = \langle \rangle$ твердження є очевидним.

Якщо ж $\mathbf{u} \neq \langle \rangle$, тоді

$$\mathbf{u} \in L \iff \mathbf{u} \cdot \langle \rangle \in L \iff \langle \rangle \in \mathbf{u}^{-1} \cdot L.$$

Тут остання еквівалентність забезпечується Твердженням 2. $\square$

Останнє твердження дозволяє звести задачу визначення чи належить ланцюжок $u \in \Sigma^*$ мові $L \subset \Sigma^*$ до задачі визначення чи належить порожній ланцюжок мові, яка є певним перетворенням мови L , тобто воно обґрунтовує коректність такого алгоритму

```
def recognize( $\Sigma$ : алфавіт,  $u$ :  $\Sigma^*$ ,  $L$ : мова над  $\Sigma$ )  $\rightarrow$  bool:
```

```
     $W = L$ 
    for  $a$  in  $u$ :
         $W = a^{-1} \cdot W$ 
    return  $\langle \rangle \in W$ 
```

5 Регулярні вирази і регулярні мови

Важливим класом мов є клас так званих регулярних мов.

5.1 Основні поняття

Визначення 2. Клас регулярних мов $\mathcal{R}$ є найменшим класом мов таким, що

- 1) $\mathcal{R}$ містить порожню, тривіальну та однолітерні мови;
- 2) $\mathcal{R}$ разом з мовою L містить також мову L^* ;
- 3) $\mathcal{R}$ разом з мовами L_1 та L_2 містить також мову $L_1 \cdot L_2$;
- 4) $\mathcal{R}$ разом з мовами L_1 та L_2 містить також мову $L_1 \cup L_2$.

Засобом опису регулярних мов є синтаксичні конструкції, які називають регулярними виразами. Регулярні вирази визначаються так.

Визначення 3. Регулярним виразом є

- 1) $\emptyset$, ε та a для будь-якого $a \in \Sigma$;
- 2) e^* якщо e є регулярним виразом;
- 3) $(e_1 \cdot e_2)$ якщо e_1 та e_2 є регулярними виразами;
- 4) $(e_1 \mid e_2)$ якщо e_1 та e_2 є регулярними виразами;
- 5) ніяких інших регулярних виразів немає.

Співставимо кожному регулярному виразу e підмножину $\llbracket e \rrbracket \subset \Sigma^*$ у такий спосіб:

1. $\llbracket \emptyset \rrbracket = \emptyset$, $\llbracket \varepsilon \rrbracket = E$ та $\llbracket a \rrbracket = L_a$;
2. $\llbracket e^* \rrbracket = \llbracket e \rrbracket^*$;
3. $\llbracket e_1 \cdot e_2 \rrbracket = \llbracket e_1 \rrbracket \cdot \llbracket e_2 \rrbracket$;
4. $\llbracket e_1 \mid e_2 \rrbracket = \llbracket e_1 \rrbracket \cup \llbracket e_2 \rrbracket$.

За визначенням, очевидно, $\llbracket e \rrbracket$ є регулярною мовою.

Нетривіальним є зворотнє твердження.

Теорема 1. Для будь-якої регулярної мови $L \subset \Sigma^*$ вірною є рівність $\llbracket e \rrbracket = L$ для якогось регулярного виразу e .

Доведення цієї теореми є достатньо громіздким і далеким від завдань цього курсу. $\square$

5.2 Спрощення регулярних виразів

Якщо є два регулярні вирази e_1 та e_2 , тоді можна говорити про рівність $e_1 = e_2$, маючи на увазі абсолютну однаковість. Однак частіше нас цікавить не ця рівність, а співпадіння відповідних мов: $\llbracket e_1 \rrbracket = \llbracket e_2 \rrbracket$.

Тут, ми наводимо декілька правил, що зменшують розмір регулярних виразів, не змінюючи мову, що відповідає цьому виразу:

1. $\emptyset * \longrightarrow \epsilon$; $\epsilon * \longrightarrow \epsilon$; $e * * \longrightarrow e*$ для будь-якого регулярного виразу e ,
2. $(\emptyset \cdot e) \longrightarrow \emptyset$; $(e \cdot \emptyset) \longrightarrow \emptyset$; $(\epsilon \cdot e) \longrightarrow e$; $(e \cdot \epsilon) \longrightarrow e$ для будь-якого регулярного виразу e ,
3. $(\emptyset \mid e) \longrightarrow \emptyset$; $(e \mid \emptyset) \longrightarrow \emptyset$ для будь-якого регулярного виразу e .

Твердження 4. Якщо регулярний вираз e' отриманий з виразу e шляхом заміни підвиразу v в e за допомогою одного з вище наведених правил, тоді $\llbracket e' \rrbracket = \llbracket e \rrbracket$.

Доведення проводиться індукцією по виразу e . □

5.3 Диференціювання регулярних виразів за Бжозовськи

Тепер покажемо, що диференціювання регулярної мови за Бжозовськи дає регулярну мову, причому регулярний вираз для цієї мови можна вказати явно.

Спочатку визначимо такі формальні правила (тут $a \in \Sigma$, а e_1 та e_2 є регулярними виразами):

1. $a^{-1} \cdot \emptyset = \emptyset$ та $a^{-1} \cdot \epsilon = \emptyset$;
2. $a^{-1} \cdot b = \begin{cases} \epsilon & \text{якщо } a = b \\ \emptyset & \text{якщо } a \neq b \end{cases}$, де $b \in \Sigma$;
3. $a^{-1} \cdot e * = ((a^{-1} \cdot e) \cdot e *)$;
4. $a^{-1} \cdot (e_1 \cdot e_2) = (((a^{-1} \cdot e_1) \cdot e_2) \mid (E(e_1) \cdot (a^{-1} \cdot e_2)))$ де $E(e) = \epsilon$, якщо $\langle \rangle \in \llbracket e \rrbracket$, та $E(e) = \emptyset$ у іншому випадку;
5. $a^{-1} \cdot (e_1 \mid e_2) = ((a^{-1} \cdot e_1) \mid (a^{-1} \cdot e_2))$.

Твердження 5. Припустимо, що e є будь-яким регулярним виразом, $a \in \Sigma$, а e' побудовано з e шляхом застосування правил наведених вище, тоді $\llbracket e' \rrbracket = a^{-1} \cdot \llbracket e \rrbracket$.

Доведення проводиться індукцією по e . □

Зауваження 1. Використана у п. 4 функція обчислюється так

$$E(e) = \begin{cases} \emptyset & \text{якщо } e = \emptyset \\ \epsilon & \text{якщо } e = \epsilon \\ \emptyset & \text{якщо } e = a \text{ для } a \in \Sigma \\ \epsilon & \text{якщо } e = e_1 * \\ \epsilon & \text{якщо } e = (e_1 \cdot e_2) \text{ та } E(e_1) = E(e_2) = \epsilon \\ \emptyset & \text{якщо } e = (e_1 \cdot e_2) \text{ та } E(e_1) = \emptyset \text{ або } E(e_2) = \emptyset \\ \epsilon & \text{якщо } e = (e_1 \mid e_2) \text{ та } E(e_1) = \epsilon \text{ або } E(e_2) = \epsilon \\ \emptyset & \text{якщо } e = (e_1 \mid e_2) \text{ та } E(e_1) = \emptyset \text{ або } E(e_2) = \emptyset \end{cases}$$

Тепер абстрактний алгоритм `recognize()` може бути реалізований для випадку регулярних мов.